

Day 01

AI & LLM Foundation

Vibe Coding 1: Setup & Foundation + LAB 1: LLM API Exploration

Ha Noi 23.07.2026 - @ Vin University



Võ Tự Đức

Google Developer Expert // ContentDrive.app



Nguyễn Minh Triết

Chapter Lead Advisor // GDG on Campus FPT University

Quét mã QR hoặc điền link:

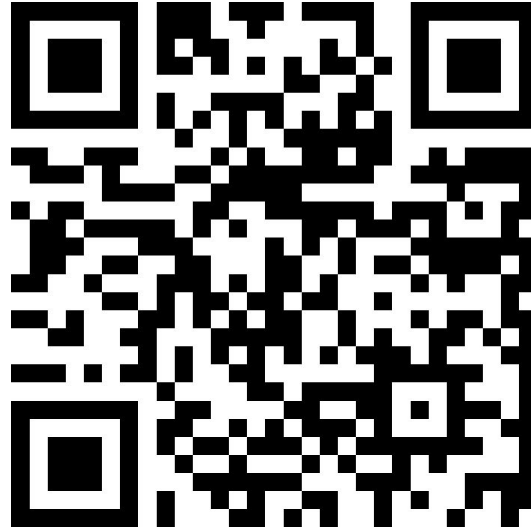
wf: VinUni AI.20K

ps: AI20K_500



<https://discord.gg/hnS86GsgW6>

Happy to know you // Slido



<https://app.sli.do/event/ibGSLQkffKbkJE5QpvD8Gm>

Agenda | Day 01

Time	Activities
14:00 - 14:45	 Lý thuyết về AI - LLM
14:45 - 15:00	 Self-attention Demo on Google Colab
15:00 - 15:15	 Nghỉ giải lao
15:15 - 15:30	 Lý thuyết về Token Economic
15:30 - 16:30	 Hands-on Lab: LLM API Exploration
16:30 - 17:00	 Q&A, Tổng kết & Homework

Live Demo: Self-attention



https://colab.research.google.com/drive/1KwOIXDb_9sFj19NXMBj-6N3PDlxV3d9I?usp=sharing

Tổng quan Codelab (Codelab Overview)

- Triển khai tích hợp các dịch vụ LLM API từ 3 nhà cung cấp hàng đầu: OpenAI, Google Gemini, và Anthropic.
- Phát triển ứng dụng Chatbot thời gian thực (Streaming) kết hợp quản lý bộ nhớ lịch sử hội thoại (Conversation History).
- Tính toán chi phí gọi API thực tế trên mỗi triệu tokens (1M tokens) cho từng dòng mô hình.
- Hiểu sâu hơn về các siêu tham số quan trọng (temperature, top_p, max_tokens) điều khiển tính sáng tạo và độ dài của kết quả LLM.

Global Checklist

- Kiểm tra email của mình đã nằm trong danh sách đăng ký của BTC. (Quan trọng vì bạn sẽ phải dùng email này để vào Classroom trong phần Setup Bài Lab)
- Kiểm tra tài khoản GitHub đã liên kết với Email đã đăng ký. (Nếu chưa có tài khoản Github, hãy tạo ngay)
- Đăng nhập và lấy [Gemini API key](#) và [OpenAI API key](#).
- [Tham gia Discord để được hỗ trợ ngay qua link bên dưới.](#)

Local Checklist

- Trên máy đã cài đặt Git (Mở cmd chạy **git version** để kiểm tra)
- Trên máy đã cài đặt Python < 3.14 & > 3.11 (Mở cmd chạy **python --version** để kiểm tra)
- Trên máy đã có IDE Visual Studio Code (VSCode).
- Đã cài đặt Extension GitHub Classroom trên VSCode.
- Đã đăng nhập vào GitHub trên VSCode

Hướng dẫn Setup bài Lab

Bước 1

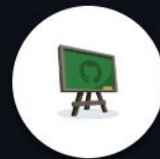
Quét mã để vào link Lab



<https://classroom.github.com/a/kq-Ti2Rb>

Bước 2

Đăng nhập vào github
(Đăng ký nếu chưa có)



Sign in to GitHub
to continue to GitHub Classroom

Username or email address

Password

[Forgot password?](#)

Sign in

or

 Continue with Google

 Continue with Apple

New to GitHub? [Create an account](#)

[Sign in with a passkey](#)

Join the classroom:

Day 10 - Data Pipeline

To join the GitHub Classroom for this course, please select yourself from the list below to associate your GitHub account with your school's identifier (i.e., your name, ID, or email).

Can't find your name? [Skip to the next step →](#)



Bước 3

Chọn Skip to the next step



Bước 4

- Kiểm tra email gửi từ github-classroom (check cả Spam và Trash nếu chưa thấy)
- Nhấn view invitation và chọn Accept

GitHub



@github-classroom[bot] has invited you to collaborate on the
[GDGoC-FPTU/data-pipeline-observability-dinhhungng](#) repository

You can [accept or decline](#) this invitation. You can also head over to <https://github.com/GDGoC-FPTU/data-pipeline-observability-dinhhungng> to check out the repository or visit [@github-classroom\[bot\]](#) to learn a bit more about them.

This invitation will expire in 7 days.



[View invitation](#)

Note: This invitation was intended for dinhhung.gect@gmail.com. If you were not expecting this invitation, you can ignore this email. If @github-classroom[bot] is sending you too many emails, you can [block them or report abuse](#).

 **llm-foundation-dinhhungng** Public

Watch 0

Fork 0

Star 0

forked from [GDGoC-FPTU/gdgoc-fptu-vinuni-day01-lab1-llm-foundation-VinUni_Codelab_Day01_Template-2](#)

main

1 Branch

0 Tags

Go to file

Add file

<> Code

This branch is **2 commits ahead** of [GDGoC-FPTU/gdgoc-fptu-vinuni-day01-lab1-llm-foundation-VinUni_Codelab_Day01_Template-2:main](#).

Contribute

Sync fork

 github-classroom[bot] add online IDE url ✓	8d41fdf · 16 hours ago	🕒 4 Commits
📁 .github/workflows	GitHub Classroom Autograding Workflow	16 hours ago
📁 starter-code	Initial commit	16 hours ago
📁 tests	Initial commit	16 hours ago
📄 .gitignore	Initial commit	16 hours ago
📄 README.md	add online IDE url	16 hours ago
📄 requirements.txt	Initial commit	16 hours ago

📖 README

About

gdgoc-fptu-vinuni-day01-lab1-llm-foundation-VinUni_Codelab_Day01_Template-2 created by GitHub Classroom

📖 Readme

🔗 Activity

📋 Custom properties

☆ 0 stars

👁 0 watching

🍴 0 forks

Report repository

Releases

No releases published
[Create a new release](#)

Packages

Bước 5

Sau khi Accept thì đây là repo của bài Lab

 agent_simulation.py	Initial commit	7 hours ago
 experiment_report.md	Initial commit	7 hours ago
 generate_garbage.py	Initial commit	7 hours ago
 raw_data.json	Initial commit	7 hours ago
 solution.py	Initial commit	7 hours ago

 README  

 Open in Visual Studio Code 

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors

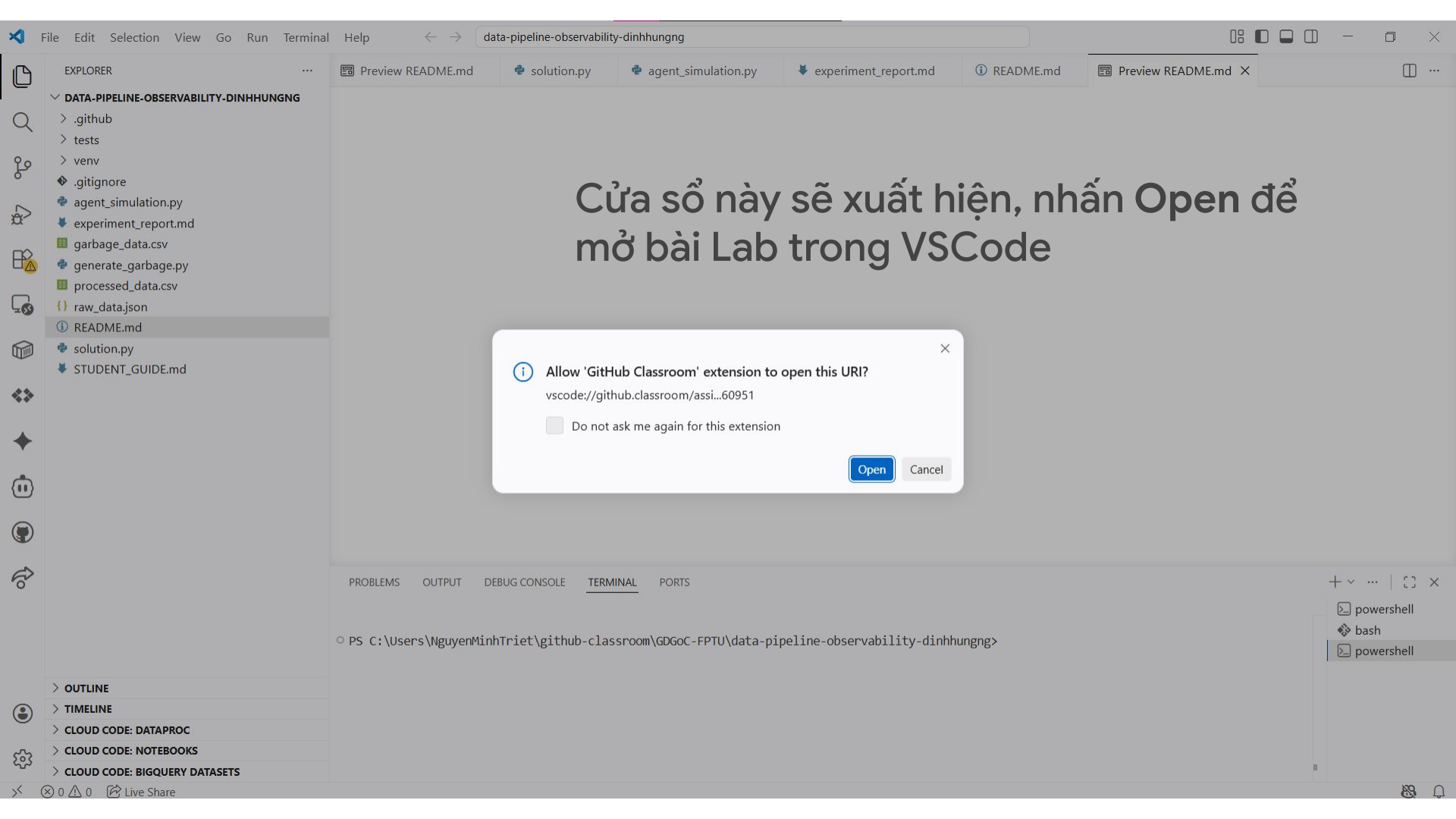
No contributors

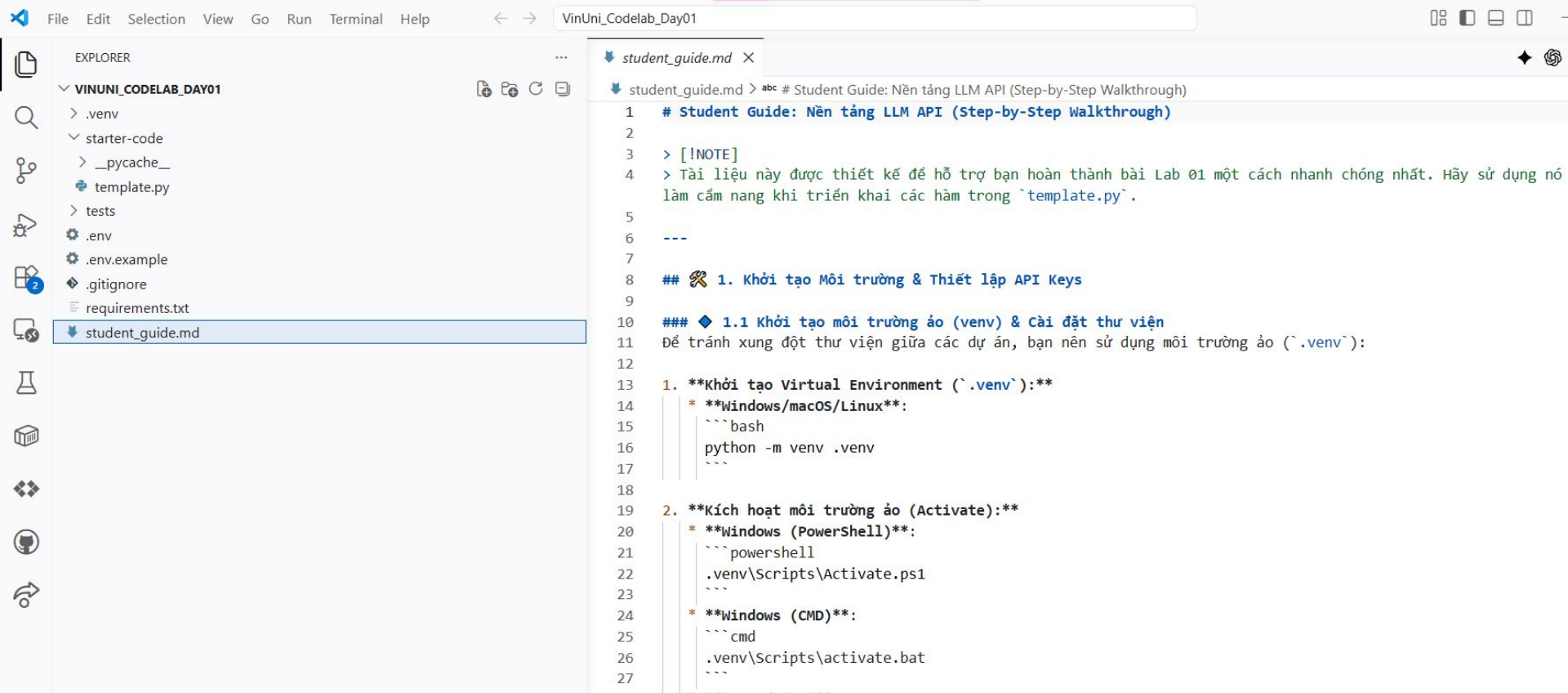
Languages

Python 100.0%

Bước 6

Kéo xuống và chọn Open in Visual Studio Code. Hoặc Clone về.
(Lưu ý bước này bạn cần có VSCode trên máy tính của mình)





Đến bước này thì Bạn đã hoàn thành phần Setup bài Lab

FileEditSelectionViewGoRunTerminalHelp

VinUni_Codelab_Day01

EXPLORER

VINUNI_CODELAB_DAY01

> .env

> starter-code

> __pycache__

template.py

> tests

.env

.env.example

.gitignore

requirements.txt

student_guide.md

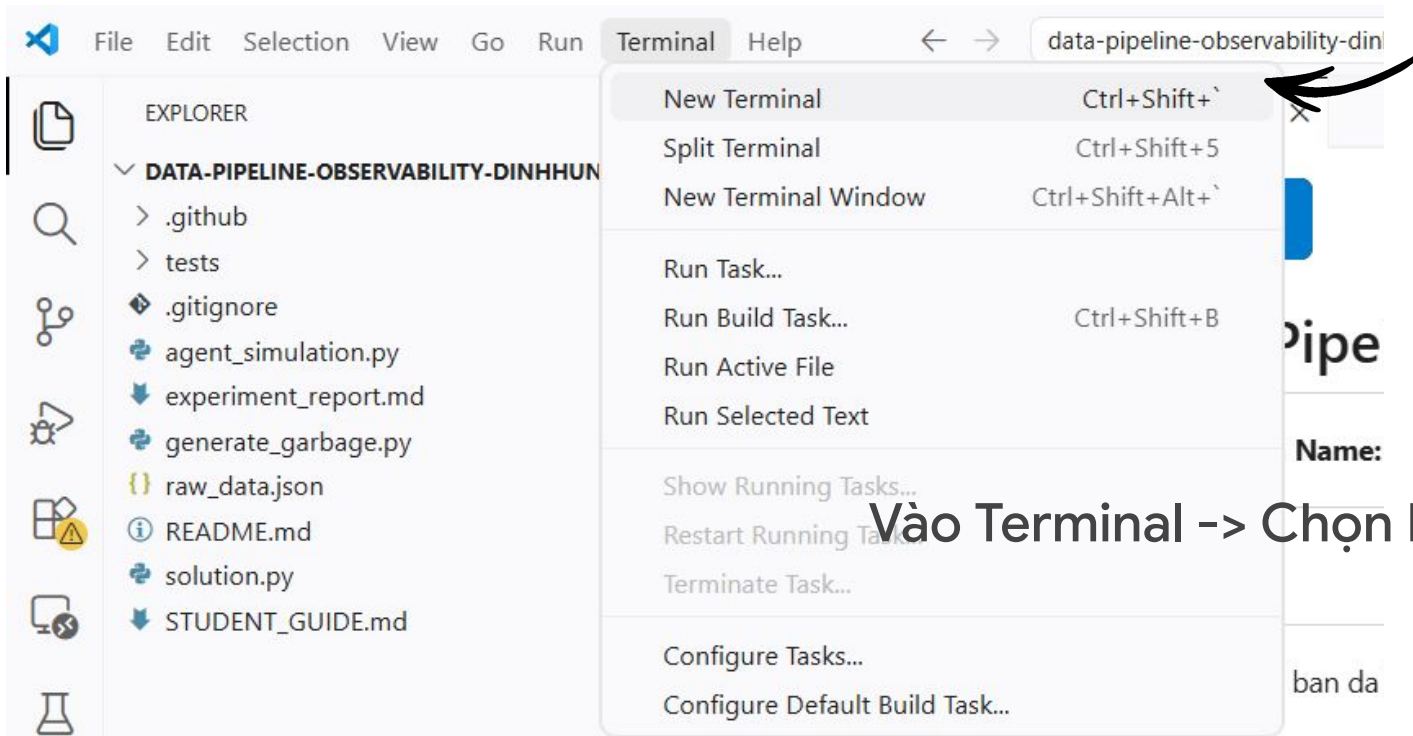
Tổng quan các Files:

template.py: File thực hành

.env: File setup biến môi trường

student_guide.md: File hướng dẫn

Hướng dẫn hoàn thành bài Lab (1)



Vào Terminal -> Chọn New Terminal

Cách chạy (How to Run)

Hướng dẫn hoàn thành bài Lab (2)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

powershell

PS C:\Users\NguyenMinhTriet\github-classroom\GDGoC-FPTU\data-pipeline-observability-dinhhungng>

Trong terminal, nhập các lệnh sau để cài đặt môi trường và thư viện



```
python -m venv venv  
.\venv\Scripts\activate  
pip install -r requirements.txt
```

Hướng dẫn hoàn thành bài Lab (3)

Kết quả khi chạy thành công Terminal sẽ hiển thị như hình:

```
Installing collected packages: pytz, tzdata, typing-extensions, tomli, six, pygments, pluggy, packaging, numpy, iniconfig,
ateutil, exceptiongroup, pytest, pandas
Successfully installed colorama-0.4.6 exceptiongroup-1.3.1 iniconfig-2.3.0 numpy-2.2.6 packaging-26.0 pandas-2.3.3 pluggy-1
.0 pytest-9.0.3 python-dateutil-2.9.0.post0 pytz-2026.1.post1 six-1.17.0 tomli-2.4.1 typing-extensions-4.15.0 tzdata-2026.1

[notice] A new release of pip available: 22.3.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS C:\Users\NguyenMinhTriet\github-classroom\GDGoC-FPTU\data-pipeline-observability-dinhhungng> █
```

Khi đó chúng ta có thể chạy được các file Python ở các bước bên dưới
Lưu ý! Nếu không thể chạy các bước trên hoặc bị lỗi không ra kết quả như trên hãy liên hệ hỗ trợ ngay trên Discord!

Hướng dẫn hoàn thành bài Lab (4)

Trong file template.py, lần lượt hoàn thành các Tasks

Thay thế toàn bộ code bên dưới để hoàn thành **Task 1**

```
# -----  
# Task 1 - Call OpenAI (GPT-4o)  
# -----  
def call_openai(  
    prompt: str,  
    model: str = OPENAI_MODEL,  
    temperature: float = 0.7,  
    top_p: float = 0.9,  
    max_tokens: int = 256,  
    ) -> tuple[str, float, dict]:  
    """  
    Call the OpenAI Chat Completions API and return the  
    and token usage stats.
```

Args:

prompt: The user message to send.
model: The OpenAI model to use (default:
temperature: Sampling temperature (0.0 - 2.0).
top_p: Nucleus sampling threshold.
max_tokens: Maximum number of tokens to generate

Returns:

A tuple of:
- response_text (str)
- latency_seconds (float)
- usage (dict with keys: 'input_tokens', 'output_tokens')

Hint:

```
from openai import OpenAI  
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))  
# response.usage contains input_tokens and output_tokens  
"""
```

```
def call_openai(  
    prompt: str,  
    model: str = OPENAI_MODEL,  
    temperature: float = 0.7,  
    top_p: float = 0.9,  
    max_tokens: int = 256,  
    ) -> tuple[str, float, dict]:  
    from openai import OpenAI  
  
    api_key = os.getenv("OPENAI_API_KEY")  
    if not api_key:  
        api_key = "mock-key"  
  
    client = OpenAI(api_key=api_key)  
  
    start_time = time.time()  
    response = client.chat.completions.create(  
        model=model,  
        messages=[{"role": "user", "content": prompt}],  
        temperature=temperature,  
        top_p=top_p,  
        max_tokens=max_tokens,  
    )  
    end_time = time.time()  
    latency = end_time - start_time  
    return response.choices[0].message.content, latency, response.usage.to_dict()
```

Hướng dẫn hoàn thành bài Lab (5)

Thay thế toàn bộ code bên dưới để
hoàn thành **Task 2**

```
# -----  
# Task 2 – Call Google Gemini 2.5 (Standard Practical Model)  
# -----
```

```
def call_gemini(  
    prompt: str,  
    model: str = GEMINI_MODEL,  
    temperature: float = 0.7,  
    top_p: float = 0.9,  
    max_tokens: int = 256,  
    ) -> tuple[str, float, dict]:  
    """  
    Call the Google Gemini API (using Gemini 2.5 Flash as sta  
    the response text, latency, and token usage stats.
```

Args:

prompt: The user message to send.
model: The Gemini model to use (default: gemi
temperature: Sampling temperature.
top_p: Nucleus sampling threshold.
max_tokens: Maximum number of tokens to generate.

Returns:

A tuple of:
- response_text (str)
- latency_seconds (float)
- usage (dict with keys: 'input_tokens', 'output_

Hint:

Option A (New Google GenAI SDK):
from google import genai
from google.genai import types
client = genai.Client(api_key=os.getenv("GEMINI_A
Configure using types.GenerateContentConfig

```
def call_gemini(  
    prompt: str,  
    model: str = GEMINI_MODEL,  
    temperature: float = 0.7,  
    top_p: float = 0.9,  
    max_tokens: int = 256,  
    ) -> tuple[str, float, dict]:  
    """  
    Call the Google Gemini API (using Gemini 2.5 Flash as standard) and return  
    the response text, latency, and token usage stats.  
  
    Supports dual-import fallback (new google-genai and legacy google-generativeai)  
    to ensure zero-friction execution.  
    """  
    api_key = os.getenv("GEMINI_API_KEY") or os.getenv("GOOGLE_API_KEY") or "mock-key"  
    start_time = time.time()  
  
    try:  
        # Option A: New Google GenAI SDK (preferred standard)  
        from google import genai  
        from google.genai import types  
  
        client = genai.Client(api_key=api_key)  
        config = types.GenerateContentConfig(  
            temperature=0.7,  
            top_p=0.9,  
            max_tokens=max_tokens,  
            response_mime_type="text/plain",  
            response_modalities="text"
```

Hướng dẫn hoàn thành bài Lab (6)

Thay thế toàn bộ code bên dưới để
hoàn thành **Task 3**

```
# -----  
# Task 3 – Call Anthropic Claude (Exploratory track)  
# -----  
def call_anthropic(  
    prompt: str,  
    model: str = ANTHROPIC_MODEL,  
    temperature: float = 0.7,  
    top_p: float = 0.9,  
    max_tokens: int = 256,  
    ) -> tuple[str, float, dict]:  
    """  
    Call the Anthropic Claude API (using Claude 3.5 Ha  
    the response text, latency, and token usage stats.  
  
    Args:  
        prompt: The user message to send.  
        model: The Claude model to use (default:  
        temperature: Sampling temperature (0.0 - 1.0).  
        top_p: Nucleus sampling threshold.  
        max_tokens: Maximum output tokens.  
  
    Returns:  
        A tuple of:  
        - response_text (str)  
        - latency_seconds (float)  
        - usage (dict with keys: 'input_tokens', '  
  
    Hint:  
        import anthropic  
        client = anthropic.Anthropic(api_key=os.getenv  
        # response.usage contains input_tokens and out  
        """  
  
    Implement with Codex  
    # TODO: Initialize Anthropic client, create messag  
    # extract content text and usage statistics,  
    raise NotImplementedError("Implement call_anthropi
```

```
def call_anthropic(  
    prompt: str,  
    model: str = ANTHROPIC_MODEL,  
    temperature: float = 0.7,  
    top_p: float = 0.9,  
    max_tokens: int = 256,  
    ) -> tuple[str, float, dict]:  
    import anthropic  
  
    api_key = os.getenv("ANTHROPIC_API_KEY") or "mock-key"  
    client = anthropic.Anthropic(api_key=api_key)  
  
    start_time = time.time()  
    response = client.messages.create(  
        model=model,  
        max_tokens=max_tokens,  
        temperature=temperature,  
        top_p=top_p,  
        messages=[{"role": "user", "content": prompt}],  
    )  
    latency = time.time() - start_time
```


Hướng dẫn hoàn thành bài Lab (7)

Thay thế toàn bộ code bên dưới để hoàn thành **Task 4**

```
# -----
# Task 4 – Compare Models (OpenAI GPT-4o vs OpenAI Mini vs Gemini 2.5 Flash)
# -----
def compare_models(prompt: str) -> dict:
    """
    Call OpenAI (gpt-4o), OpenAI Mini (gpt-4o-mini), and Gemini 2.5 Flash (ge
    with the same prompt and return a structured comparison dictionary.

    Calculate the exact USD token cost for input + output using the prices in

    Args:
        prompt: The user message to send to all models.

    Returns:
        A dictionary containing:
        - "gpt4o": { "response": str, "latency": float, "cost": float, "i
        - "gpt4o_mini": { "response": str, "latency": float, "cost": floa
        - "gemini_flash": { "response": str, "latency": float, "cost": fl
    """
    Implement with Codex
    # TODO: Call call_openai with default gpt-4o model
    Implement with Codex
    # TODO: Call call_openai with gpt-4o-mini model
    Implement with Codex
    # TODO: Call call_gemini with default gemini-2.5-flash model
    Implement with Codex
    # TODO: Calculate costs exactly based on input and output token counts us
    #         Formula: Cost = (input_tokens * input_rate_per_1M + output_tokens
    Implement with Codex
    # TODO: Assemble and return the comparison dictionary.
    raise NotImplementedError("Implement compare_models")
```

```
def compare_models(prompt: str) -> dict:
    """
    Call OpenAI (gpt-4o), OpenAI Mini (gpt-4o-mini), and Gemini 2.5
    Flash (gemini-2.5-flash)
    with the same prompt and return a structured comparison
    dictionary.
    """
    # Call GPT-4o
    gpt4o_text, gpt4o_lat, gpt4o_usage = call_openai(prompt,
    model=OPENAI_MODEL)
    gpt4o_cost = (
        gpt4o_usage["input_tokens"] *
        PRICING_1M_TOKENS["gpt-4o"]["input"] +
        gpt4o_usage["output_tokens"] *
        PRICING_1M_TOKENS["gpt-4o"]["output"]
    ) / 1_000_000

    # Call GPT-4o-mini
    mini_text, mini_lat, mini_usage = call_openai(prompt,
```

Hướng dẫn hoàn thành bài Lab (8)

Thay thế toàn bộ code bên dưới để hoàn thành **Task 5**

```
# -----
# Task 5 – Streaming chatbot with Gemini 2.5 (Focus Model)
# -----
def streaming_chatbot() -> None:
    """
    Run an interactive streaming chatbot in the terminal using
    Gemini 2.5 Flash.

    Behaviour:
    - Streams response tokens from Gemini 2.5 Flash as they arrive.
    - Maintains the last 3 turns of conversation history for context.
    - Typing 'quit' or 'exit' ends the session.

    Hints:
    - Maintain a history list of conversation turns.
    - Check how to stream responses using client.chats or client.stream.
    - Keep history limited to the last 3 turns to optimize memory.

    Implement with Codex
    # TODO: Setup interactive session, prompt user for input,
    raise NotImplementedError("Implement streaming_chatbot")
```

```
def streaming_chatbot() -> None:
    """
    Run an interactive streaming chatbot in the terminal using Gemini
    2.5.

    Maintains the last 3 turns of conversation history for context.
    """
    api_key = os.getenv("GEMINI_API_KEY") or
os.getenv("GOOGLE_API_KEY")
    if not api_key:
        print("\033[93m[System Warning] GEMINI_API_KEY environment
variable not set. Running in dummy mode.\033[0m")
        api_key = "mock-key"

    print("\n\033[94m=====")
    print("🤖 Vin Smart Future – Intelligent Chat Assistant")
    print("Powered by Google Gemini 2.5 Flash")
    print("Type 'quit' or 'exit' to end the session.")
    print("=====\033[0m\n")
```

Hướng dẫn hoàn thành bài Lab (9)

Optional: Vibe Code để hoàn thành các Bonus Tasks

```
# -----  
# Bonus Task A – Retry with exponential backoff  
# -----  
def retry_with_backoff(  
    fn: Callable[[], Any],  
    max_retries: int = 3,  
    base_delay: float = 0.1,  
) -> Any:  
    """  
    Call fn(). If it raises an exception, retry up to max_retries times  
    with exponential backoff (delay = base_delay * 2^attempt).  
    """  
    last_exception = None  
    for attempt in range(max_retries + 1):  
        try:  
            return fn()  
        except Exception as e:  
            last_exception = e
```

Hướng dẫn hoàn thành bài Lab (10)

Trong Terminal, gõ lệnh sau để test kết quả

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

powershell

PS C:\Users\NguyenMinhTriet\github-classroom\GDGoC-FPTU\data-pipeline-observability-dinhhungng>

python starter-code/template.py

Hướng dẫn hoàn thành bài Lab (11)

Kết quả sẽ như hình và có thể tương tác được:

```
=== Starting Gemini 2.5 Chatbot (type 'quit' to exit) ===
```

```
=====
🤖 Vin Smart Future – Intelligent Chat Assistant
Powered by Google Gemini 2.5 Flash
Type 'quit' or 'exit' to end the session.
=====
```

```
You: Hello
```

```
Gemini: Hello! How can I help you today?
```

Sau khi chạy thành công, hãy trả lời các câu hỏi trong `exercises.md`

Lưu ý! Nếu không thể chạy các bước trên hoặc bị lỗi hãy liên hệ hỗ trợ ngay trên Discord!

Hướng dẫn nộp bài và xem Result (1)

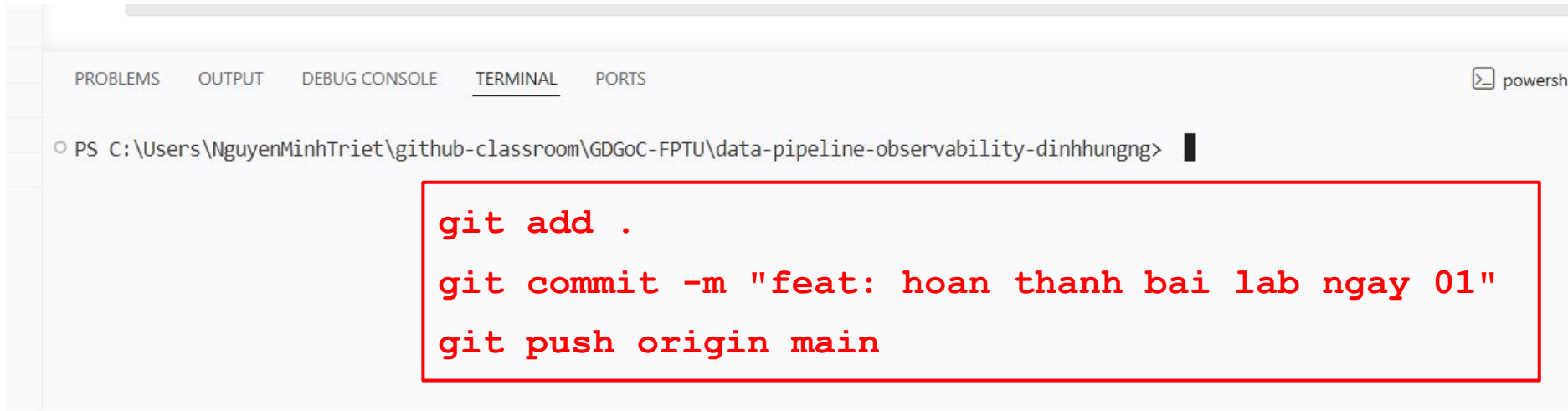
Trước khi nộp bài, tạo file `student_info.md` ghi:

- Họ và Tên
- Email cá nhân đăng ký chương trình
- GitHub Username
- Discord Username đã tham gia discord server

Lưu ý! Nếu quá trình này gặp lỗi, hãy liên hệ ngay trên Discord

Hướng dẫn nộp bài và xem Result (2)

Mở Terminal và nhập lệnh sau để nộp bài:



The image shows a screenshot of a Visual Studio Code terminal window. The terminal has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active. The prompt shows the current directory is C:\Users\NguyenMinhTriet\github-classroom\GDGoC-FPTU\data-pipeline-observability-dinhhungng. A red box highlights the following Git commands:

```
git add .  
git commit -m "feat: hoan thanh bai lab ngay 01"  
git push origin main
```

Lưu ý! Nếu quá trình này gặp lỗi, hãy liên hệ ngay trên Discord

Hướng dẫn nộp bài và xem Result (3)

GDGoC-FPTU / data-pipeline-observability-dinhhungng

Code Issues Pull requests **Actions** Projects Wiki Security and quality Insights Settings

Actions New workflow

All workflows Filter workflow runs

Showing runs from all workflows

8 workflow runs

Workflow	Event	Status	Branch	Actor
✓ Merge branch 'main' of https://github.com/GDGoC-FPTU/data-pipeli...			main	
Autograding Tests #8: Commit 576f933 pushed by dinhhungng				
🕒 GitHub Classroom Autograding Workflow			main	
Autograding Tests #7: Commit 0a2908d pushed by github-classroom (bot)				
✗ feat: hoan thanh bai lab ngay 10			main	
Autograding Tests #6: Commit f499e60 pushed by dinhhungng				
✗ feat: hoan thanh bai lab ngay 10			main	
Autograding Tests #5: Commit dc524ce pushed by dinhhungng				
🕒 GitHub Classroom Autograding Workflow			main	
Autograding Tests #4: Commit 40b7673 pushed by github-classroom (bot)				
✗ GitHub Classroom Autograding Workflow			main	
Autograding Tests #3: Commit 74595ec pushed by github-classroom (bot)				

Trong màn hình github, vào Actions để xem điểm

Chọn run mới nhất để xem điểm